Université Bourgogne Europe

TEM Nanoparticle Segmentation using Classical Image Processing

MUHAMMAD HARIS AMJAD | SHAHMIR JAVED | HAMZA KHALID

Overview

01 Problem Statement

02 Project Objectives

03 Overall Processing Pipeline

04 Results & Analysis

05 Conclusion

Problem Statement

Key Challenges

Multiple Nanoparticles
Images contain dense clusters of particles, varying in count and arrangement.

Noise & Illumination
High sensor noise and uneven lighting gradients across the TEM field of view.

Touching Particles
Particles often touch or overlap, making individual separation difficult without advanced techniques.

Goals & Constraints

Segment Full Particles
Identify complete particles only; explicitly ignore and remove border-clipped artifacts.

Classical Methods Only
Strict constraint: No Machine Learning or Deep Learning approaches allowed.

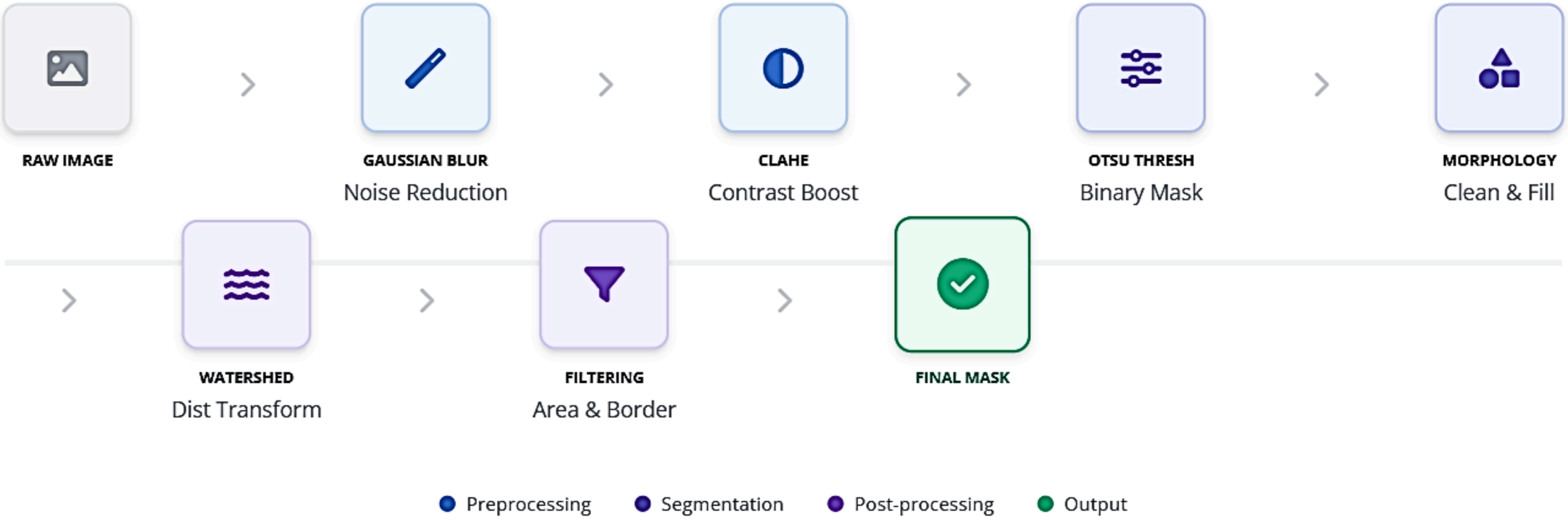
Enable Analysis
Output must facilitate downstream calculation of dimensions (axes, area) for chemists.

Project Objectives

- Robust Segmentation Pipeline
- Separate Touching Particles
- Clean & Filter Artifacts
- Quantitative Evaluation
- Clear Visualization

Data:
136 Grayscale TEM Images

Overall Processing Pipeline



Preprocessing: Gaussian Blur

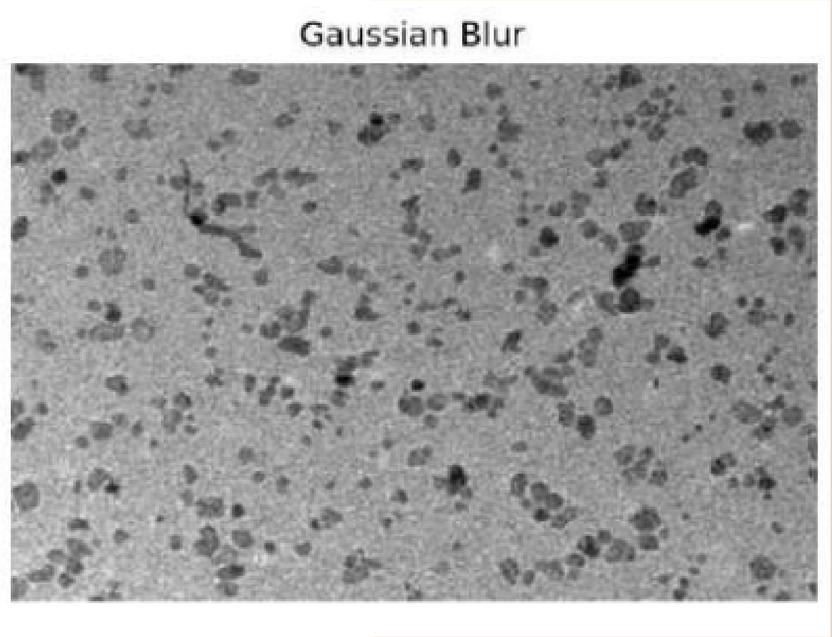
Implementation:
A kernel size of (5,5) with sigma 1.2 provided the optimal balance for TEM images.

Why Gaussian Blur?

- **Noise Reduction.**
- **Texture Smoothing.**
- **Edge Preservation.**

Why Not?

- **Median Filter:** Better for salt-and-pepper noise, but tends to blur fine edges needed for segmentation.
- **Bilateral Filter:** Preserves edges well but is computationally expensive and slower to execute.
- **Mean Filter:** Blurs edges too aggressively, causing loss of particle definition.



Contrast Enhancement: CLAHE

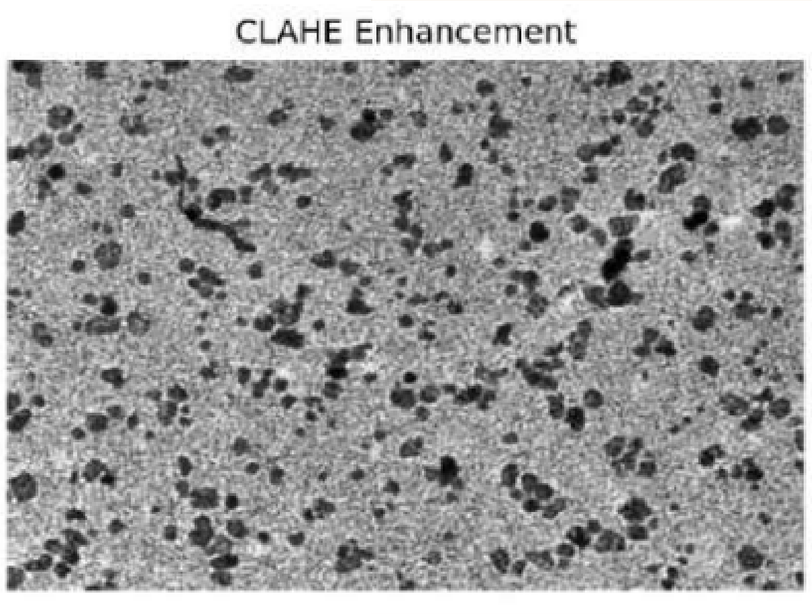
Implementation:
Adaptive histogram equalization with a clip limit of 2.5 prevents over-amplification of noise in flat regions..

Why CLAHE?

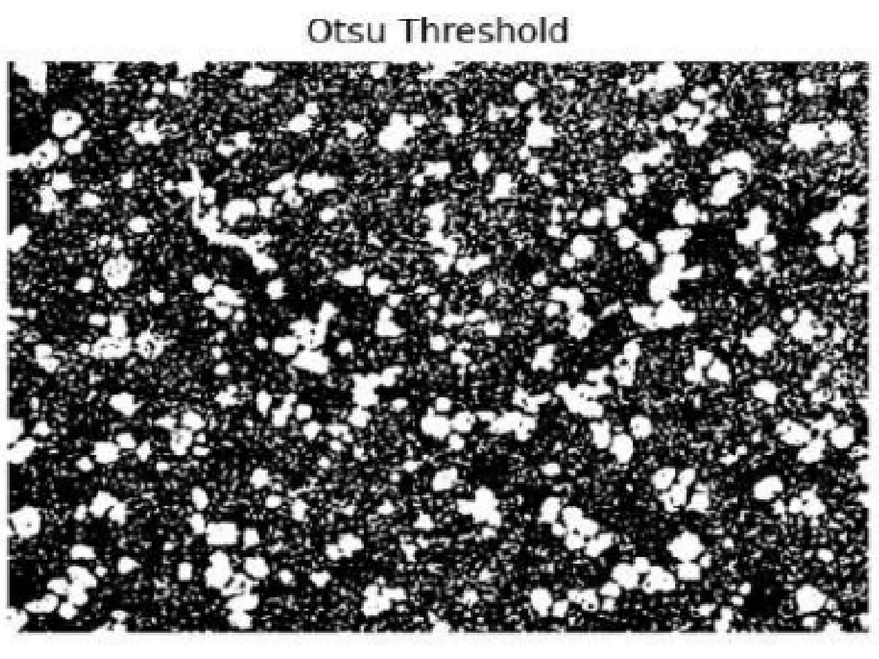
- **Uneven Illumination.**
- **Local Contrast.**
- **Noise Control.**

Why Not?

- **Global Histogram Eq:** Over-enhances background noise in flat regions, creating artifacts.
- **Gamma Correction:** Requires manual tuning per image; difficult to automate for entire dataset.
- **Linear Normalization:** Poor performance under non-uniform lighting; fails to enhance local details.



Thresholding: Otsu's Method



Implementation:
We apply Binary Inversion combined with Otsu's Algorithm to automatically determine the optimal global threshold.

Why Otsu's Method?

- **Automatic Tuning**
- **Bimodal Robustness:**
- **Global Stability:**

Why Not?

- **Fixed Threshold:** Not robust to illumination changes; requires manual tuning per image.
- **Adaptive Mean/Gauss:** Over-segments the noisy TEM background texture, creating false positives.
- **K-means Clustering:** Significantly slower and can be unstable for simple grayscale intensity separation.

Morphological Operations

Why Morphology?

- **Removes False Positives -** Eliminates tiny noise blobs that aren't particles
- **Smooths Contours -** Cleans ragged edges for better shape analysis
- **Improves Stability -** Essential prep for Distance Transform & Watershed

Applied Techniques

Opening (Noise Removal)

Erosion followed by Dilation. Removes small white noise artifacts from the black background.

```
cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
```

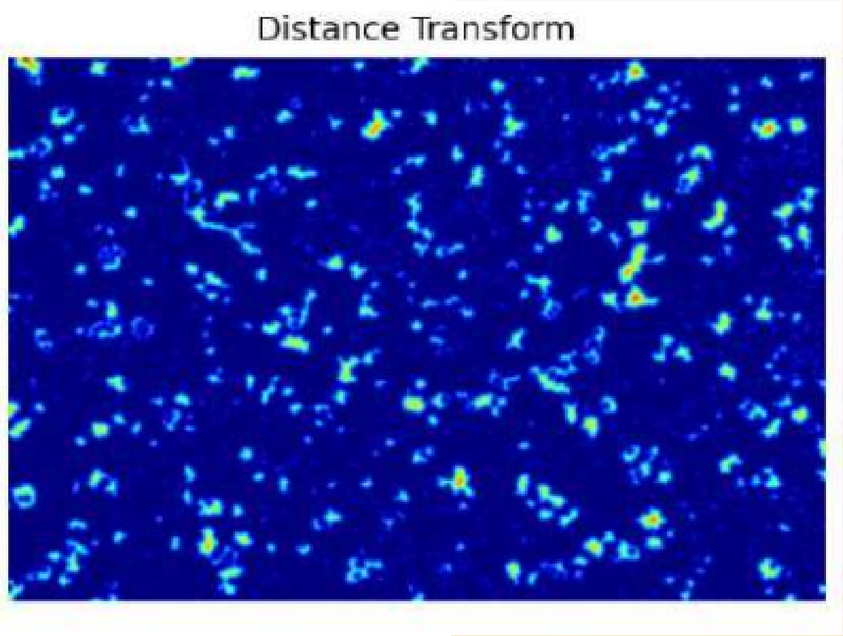
Closing (Hole Filling)

Dilation followed by Erosion. Fills small black holes inside the white foreground particles.

```
cv2.morphologyEx(opening, cv2.MORPH_CLOSE, kernel)
```

Distance Transform

Implementation:
We apply the Euclidean Distance (L2) with a 5x5 mask to convert binary particles into a topological map.



Why Distance Transform?

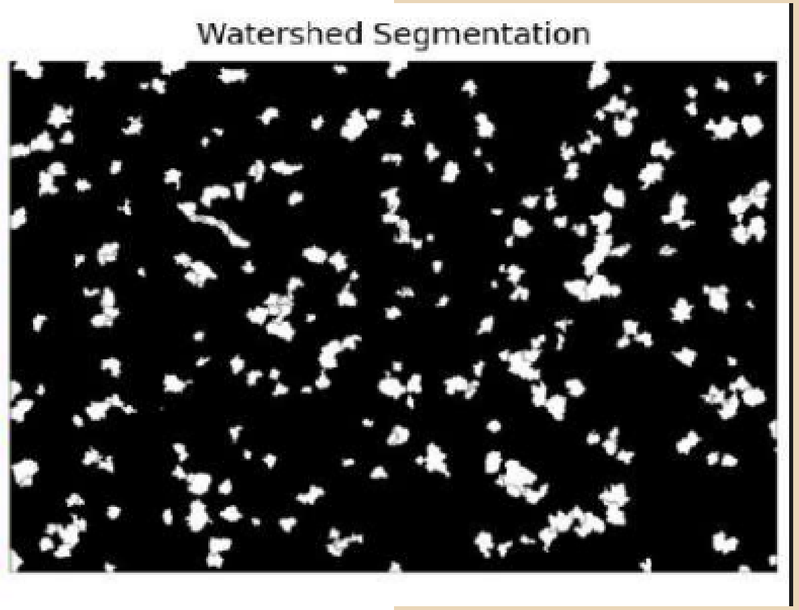
- **Watershed Markers.**
- **Touching Separation.**
- **Center Detection.**

Why Not?

- **Erosion:** Distorts shape significantly and can completely eliminate small valid particles.
- **Convex Hull:** Fails on clusters; tends to merge touching objects into a single large blob.
- **Edge Thinning:** Produces fragmented skeletons and is highly unstable in noisy backgrounds.

Watershed Segmentation

Implementation:
Floods the topographic surface defined by the distance transform from the markers, building "dams" to separate touching basins.



Why Watershed Segmentation?

- **Touching Objects.**
- **Topographic Basis.**
- **Closed Boundaries.**

Why Not?

- **Connected Comp:** Merges touching particles into single blobs; fails to distinguish individuals.
- **Contour Splitting:** Heuristics based on convexity defects are fragile and shape-dependent.
- **Hough Transform:** Only works for perfectly circular objects; fails on irregular shapes.

Particle Filtering

Strategic Rationale:

- **Project Compliance:** Explicitly satisfies the requirement to segment only "FULL" particles and ignore clipped border artifacts.
- **Noise Reduction:** Area thresholds effectively remove small sensor noise specks that survived morphological opening.
- **Efficiency & Stability:** Bounding box checks are computationally faster (O(1)) and more stable than complex shape descriptors for partial objects.

Implementation Methods

Area Filtering

`cv2.contourArea()`

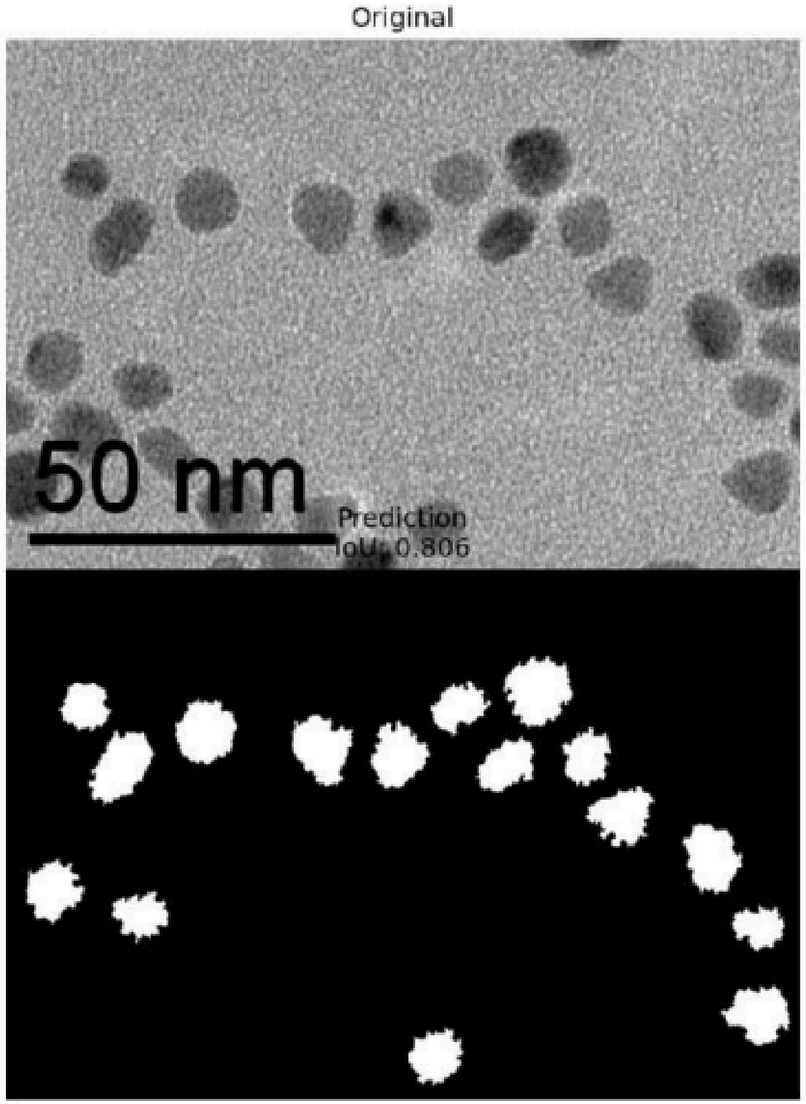
Calculates the area of each contour. Particles below a minimum threshold (noise) or above a maximum threshold (merged clusters) are discarded.

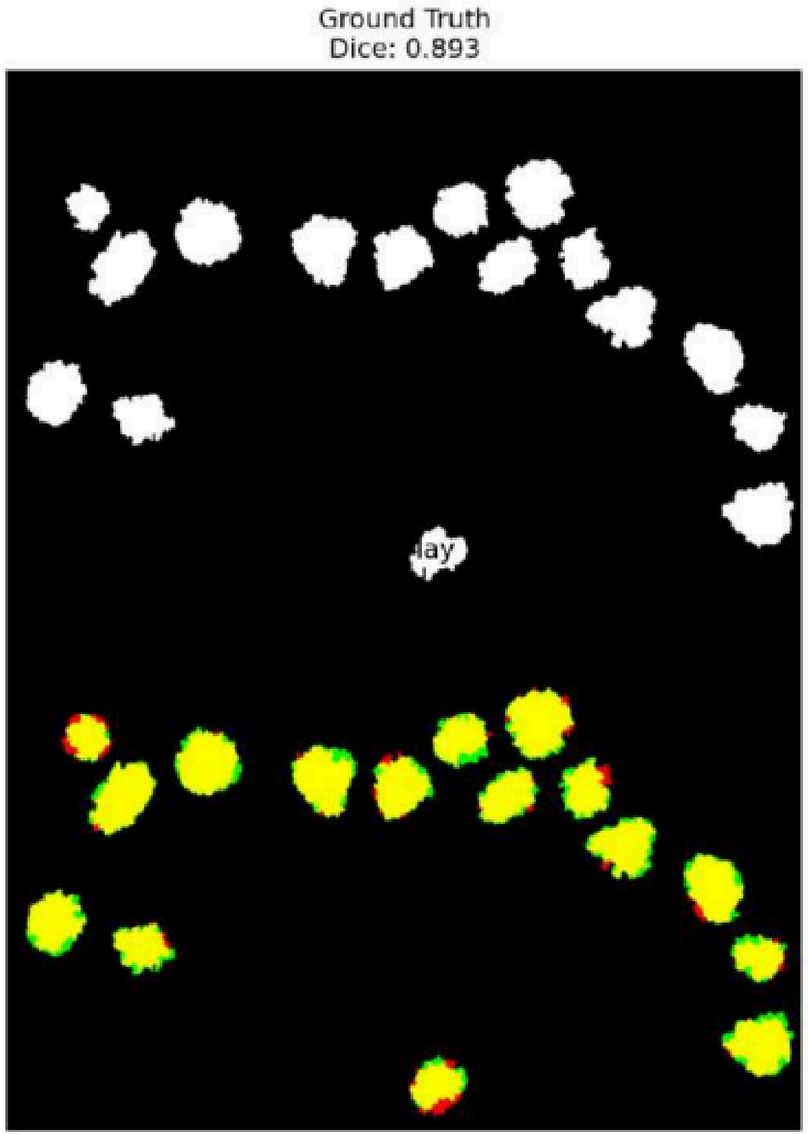
Border Exclusion

`cv2.boundingRect()`

Detects bounding boxes touching image edges (x=0, y=0, x=W, y=H). These "clipped" particles are excluded from the final mask.

Results & Analysis





Results & Analysis

Strengths:

- No Training Required
- Touching Particle Separation
- Illumination Robustness
- Border Handling
- Fast Execution

Limitations?

- Sensitive to Extreme Noise
- Irregular Shape Over-segmentation
- Parameter Sensitivity
- Cannot Learn Texture Patterns

```
================ SUMMARY ================
Images processed: 136
Mean Dice:        0.5591
Mean IoU:         0.4139
Mean Precision:   0.4953
Mean Recall:      0.7497
=========================================
```

```
PS C:\Users\LENOVO\Documents\Health AI Study\M2 SIA Image Processing
ata/Local/Programs/Python/Python313/python.exe "c:/Users/LENOVO/Docu
ge processing Project/tem_nanoparticle_segmentation.py"
001 | Dice: 0.408 | IoU: 0.256 | precision: 0.277 | recall: 0.773
002 | Dice: 0.467 | IoU: 0.305 | precision: 0.328 | recall: 0.810
003 | Dice: 0.338 | IoU: 0.204 | precision: 0.218 | recall: 0.754
004 | Dice: 0.317 | IoU: 0.188 | precision: 0.202 | recall: 0.733
005 | Dice: 0.624 | IoU: 0.454 | precision: 0.552 | recall: 0.719
006 | Dice: 0.678 | IoU: 0.512 | precision: 0.587 | recall: 0.801
007 | Dice: 0.504 | IoU: 0.336 | precision: 0.373 | recall: 0.773
008 | Dice: 0.685 | IoU: 0.520 | precision: 0.659 | recall: 0.713
009 | Dice: 0.415 | IoU: 0.262 | precision: 0.286 | recall: 0.752
010 | Dice: 0.481 | IoU: 0.316 | precision: 0.365 | recall: 0.704
```

Conclusion

- Developed a fully classical pipeline for TEM nanoparticle segmentation.
- Achieved a mean Dice score of ~0.55, reasonable given noise, overlap, and strict masks.
- Successfully separated touching particles and removed clipped objects.
- Demonstrated the effectiveness of classical image processing for nanoscale segmentation.

THANK YOU

28th January 2026